

Robots Learn Visual Pouring Task Using Deep Reinforcement Learning with Minimal Human Effort

Homayoun Moradi

Human and Robot Interaction Laboratory
School of Electrical and
Computer Engineering
University of Tehran
Tehran, Iran
homoradi@ut.ac.ir

Mehdi Tale Masouleh

Human and Robot Interaction Laboratory
School of Electrical and
Computer Engineering
University of Tehran
Tehran, Iran
m.t.masouleh@ut.ac.ir

Behzad Moshiri

School of Electrical and
Computer Engineering
College of Engineering
University of Tehran
Tehran, Iran
moshiri@ut.ac.ir

Abstract—In this paper, the entire pipeline for teaching a robot to perform the visual pouring task is addressed with a minimal human supervision. In this task the robot's arm that carrying a glass, and the target container are initialized in an arbitrary position. The reinforcement learning approach is employed without the use of reward engineering where the reward function is approximated by a Convolutional Neural Network. This network, which is trained on a little set of goal images (positive samples), takes a raw RGB image as input. In data gathering step, only goal conditions are established by an expert, and positive samples are collected in a self-supervised manner. The major contribution of this paper consists in using only the RGB image, without 3D analysis which is not possible by classical approaches, and self-supervised positive sample gathering which has not been addressed in recent RL studies. Moreover, the deep metric learning approach is utilized to train the reward network in order to obtain increased robustness against unseen negative samples. The foregoing is applied since this approach classifies the negative samples as ignore class and prevents the network from being fooled by the RL algorithm. Soft Actor-Critic is the RL algorithm employed in this study. A fixed monocular camera is the single sensor used for perception and RGB image is used as the representation of the state for action selection, value estimation, and input of reward network. The action-space is defined such a way that reduces convergence time, and in this approach, the policy network generates the desired 3D position in each step. This method of defining the action-space resulted in a reduction in training steps from 100k-1M to 1k. The provided method is compared to joint-based action-space and sparse-reward to validate the good effect of this combination of action-space and reward, i.e., position-based action-space, and continuous reward. Some experiments are designed as case studies in which a NAO H25 robot interacts with the environment and performs the visual pouring task. A CoppeliaSim simulation environment is used for both training and evaluation and the proposed method achieved the success rate 93.7 after 1k episodes for the approaching phase of visual pouring task.

Index Terms—Deep reinforcement learning (DRL), Visual pouring, Minimal human effort, Reward engineering, Soft actor-critic (SAC)

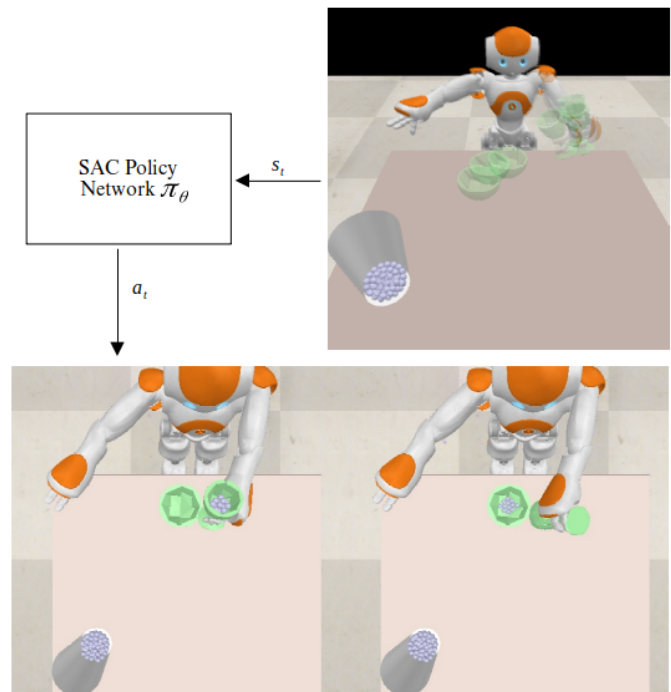


Fig. 1. In each episode, for training or evaluation, the robot's hand and target bowl are initialized in a random position (Up). Image of the current step is fed into policy network and an action is generated (Left) and then robot twists its arm to complete pouring task and successfully done by lying all spheres in target container (Right).

I. INTRODUCTION

In fact, Deep RL approaches were promising in recent years and addressed unsolved problems that generally comes from the control of robotic mechanical systems [1], embodied AI, human-robot interaction and computer games [2] domains. Training a policy which can learn the value of state and state-action pairs only based on camera images gained good attention from researchers. The design of the reward function

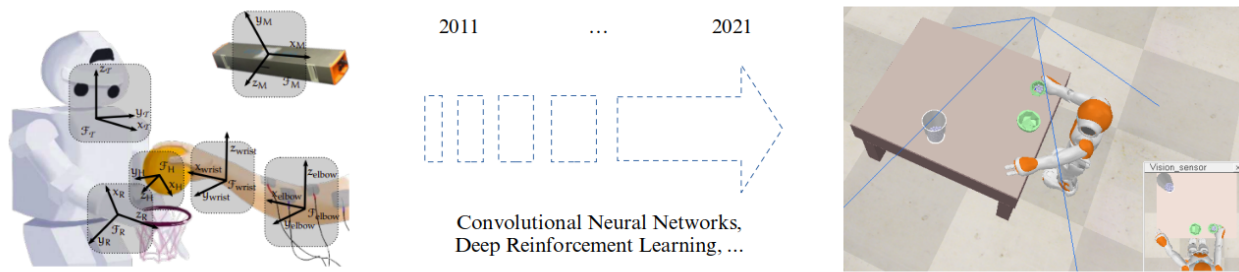


Fig. 2. Before deep neural networks and deep reinforcement learning, 3D analysis and coordination systems were utilized to solve human-robot interaction problems that required a variety of sensors (left image). However, by combining CNNs with Deep RL, these difficult tasks may be efficiently handled by a single RGB camera (right image).

is an important step to train an RL method and there is relatively limited practical guidance about this how these rewards should be designed in RL research. In fact, there are a host of consideration in defining a reward function in RL which should be done by an expert use for the under study problem. Using motion capture, depth sensors, force sensors, sound sensors, and physical markers is common in studies to address this issue and reduce the complexity of state coding [3] as well as reward engineering. As an alternative to reward specification, imitation learning and inverse reinforcement learning aim at replicating expert behavior [4] and avoid unexpected behavior which comes from improper reward function design. The newest approach for dealing with this issue is training a classifier to differentiate desired and unwanted states. This is a simple but yet effective method to put these intended results to use [5].

Unlike simulations, there is no detailed and precise information in real-world tasks and vision sensors are the most important element of the perception system. Most of the traditional techniques fail in complicated tasks without a 3D measurement of the environment, and RL-based solutions perform poorly. Furthermore, in some activities, defining a reward function is impossible or extremely complex and it might be unreliable in a noisy environment. In some circumstances, the robot should determine if the current state is a goal state or not, for example, while switching from one phase to the next in multi-step tasks.

Human demonstrations have received a lot of interest in recent researches because of their ease of gathering and the presence of a large amount of data on public venues, both organized and unstructured. Another novel option is weak supervision, in which the user gives certain labeled states, which are then utilized to train a neural network which serves as a reward function. There are still several drawbacks to this method for constructing a reward function, such as agents learning to deceive the neural network or negative sampling being difficult for users where recent studies tried to address these issues [6].

The most recent wave of effort to get robots to perform activities more like humans is centered on cognition reasoning and concepts or semantic understandings [7]. These works are

primarily powered by cutting-edge deep learning approaches which provide a decent representation for examining semantic relationships between objects [8]. Also, reducing human effort in data gathering and task execution is another topic which has received good attention in recent years [9], [10].

Visual pouring was chosen as a multi-stage real-world problem where the robot should approach the glass, pick it up [11], move it near to the target bowl, and then spill the contents of the glass into the target container in order to finish the task. This study aims at solving one difficult phase, i.e., approaching the glass near to the bowl in such a way that pouring becomes simple in which if the robot rotates its wrist the pouring task would be completed successfully. Because there is no data beyond the RGB image to solve this stage, none of the traditional methodologies such as inverse kinematics or path planning can be applied. Also, in order to improve the efficiency of the learning process, the robot tries to learn the best position close to the bowl for moving the glass rather than focusing on the end goal, i.e., pouring the contents of the glass into the target container, and the reward network is in charge of predicting whether this state can be followed by pouring or not. Figure. 1 shows a random initial state and the image of the current step which is fed into the policy network and the action which is generated by the stochastic policy that comes from a gaussian distribution with μ and σ . The robot moves its hand according to the selected position by learned policy and then twists its arm to complete the pouring task which successfully done by lying all spheres in the target container.

The reward function is a neural network which is trained using positive and negative data acquired during the random exploration period. The agent labels its states, while humans only define the final objective condition. Defined goal conditions are similar to human cognition and simply can determines whether or not the task is completed. Although, moving glass close to the bowl is used as an example and possibly one can extend it to other steps and several other real-world tasks can be formulated using the same reasoning.

In summary, the main contribution of this paper consists in formulating the visual pouring task as a self-supervised RL

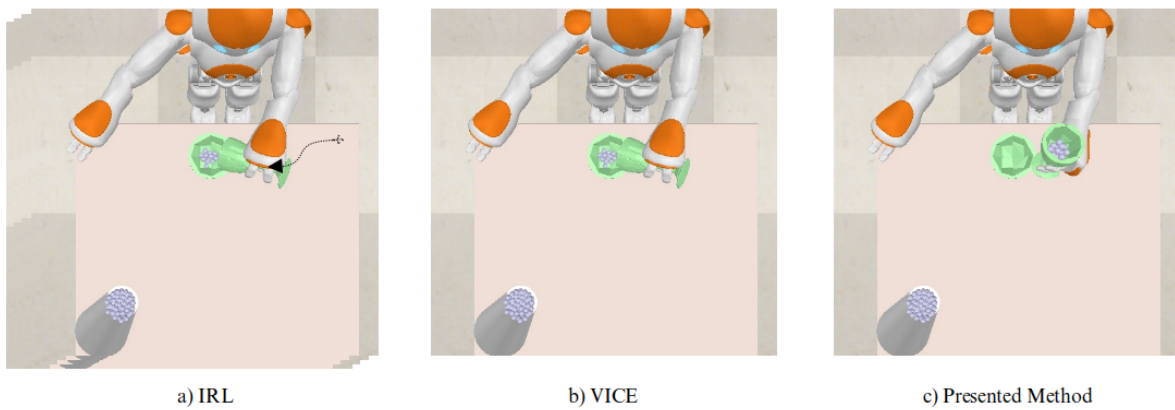


Fig. 3. Difference between presented approach, IRL and VICE in observation coding. IRL need full observation, VICE works only with final state and presented approach works with proper intention before last action to reach to the desire state.

problem without reward engineering, and using a CNN instead of a reward function which is trained by the deep metric learning objective to deliver a robust reward approximator. Moreover, the training steps are efficiently defined which leads to reduce training time and increase interpretability.

The remainder of this paper is organized as follows. Section II dives deep into related works, comparing and contrasting various techniques propounded in the literature, as well as describing their benefits and downsides. In Section III, the process of goal conditions defining, data gathering and the reward network training is described in the reward approximation subsection and then in the other subsection the Deep RL method and training steps are discussed. In Section IV, the obtained experiments are examined and compared with respect to collected reward, convergence speed, ease of use, and generalization ability. Finally, in Section V the paper is concluded by summarizing the outcomes and contribution. Moreover, some description for future works and unresolved concerns are provided in last which opens an avenue for further study.

II. RELATED WORKS

Robot manipulation has a long record in the areas of robotics and artificial intelligence. Prior to the advent of learning based methods, particularly deep reinforcement learning-based methods, researchers used classical methodologies and environment-specific approaches to enable robots to accomplish certain tasks. For many years, inverse kinematics and path planning constituted the foundation of this literature. The primary disadvantages of these approaches were their time, lack of robustness, and inability to be learn [12]. Furthermore, the mathematical method for addressing this task is quite complex and is dependent on a specific environment which has already been explored by several studies [13]. In fact, this task was solved using 3D analysis of involved components which is depicted in Figure. 2. Learning from human demonstrations was the next move in the robotics field's effort to include learning approach in order to develop more intelligent robots

which could learn from people. Demonstrations are divided into two main categories, namely, visual demonstrations and motor-control systems. In addition, the level of information coding can be low-level like trajectory coding or high-level like a sequence of meaningful states or actions [14].

The nonlinear characteristic of neural networks was used to code complicated states, such as pictures, which could not be coded by hand-crafted features, allowing researchers to add strong perceptual gadgets to their robots. After merging with deep learning, reinforcement learning with a solid theoretical backbone has demonstrated excellent results and surpassed earlier approaches [15]. The sophistication of the reward function designing, which was still a manual procedure, was the major disadvantage of RL techniques in robot manipulation. Imitation learning and Inverse Reinforcement Learning (IRL) techniques were developed to solve the latter issue. The significant difficulty of data gathering and the non-efficient learning process may rich the learning process to several million episodes were some of the disadvantages.

Recent research has mostly focused on data efficiency, with some attempts to develop solutions which would eliminate the need for complete process demonstrations. Variational Inverse Control with Events (VICE) [16] was focused on well-developed frameworks which handle this issue simply by utilizing goal information rather than entire process demonstrations. Following that, various studies address VICE's shortcomings, such as the potential of network tricking or the difficulties of negative sampling [6]. Another aspect which nowadays stimulated the attention of many researchers is the time efficiency of learning approaches. The combination of a high degree of freedom and multi-step tasks which should be completed to reach the final objective, as well as the robot's lack of knowledge of the semantics of the environment, causes the robot to spend a lot of time to explore, causing to increase the convergence time [17].

The pouring task has been studied in artificial intelligence and robotics literature from a variety of perspectives, with researchers focusing on characteristics of pouring such as

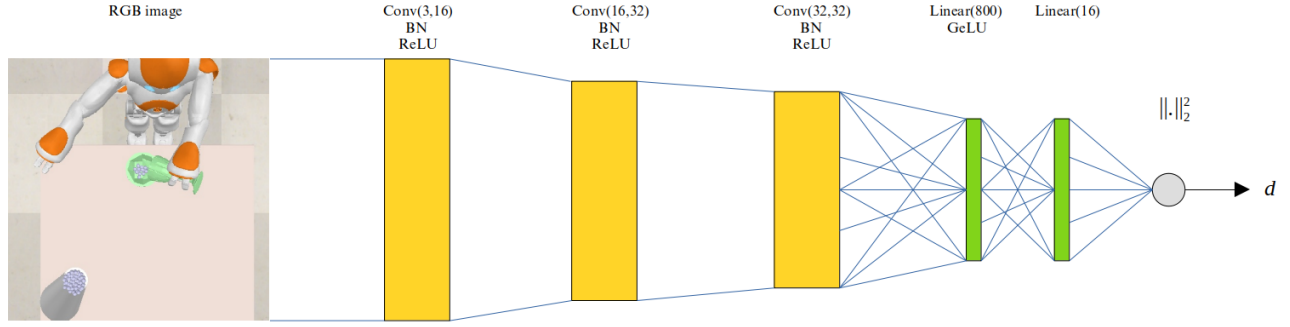


Fig. 4. The deep-RBF architecture with the rejection option utilized at the end of the CNN feature extractor. The network's output is the distance (d) between the sample (x_i) and the positive class, which is compared to a threshold for the classification task and utilized as a reward in learning duration.

accuracy in amount [3], smoothness, and the measure of acting like a human. The majority of demonstrations for this task are motor-control demonstrations [18], which allow an expert to move the robot end-effector to perform the pouring task, but visual demonstrations have grown in popularity in recent years, demonstrating that robots can learn to imitate human-generated trajectories in these situations. Because data collecting is a challenging task, researchers often confine pouring tasks to a certain configuration, which reduces generalization of this sort of learning or need more information like depth sensors [19]. This problem can be solved by combining new breakthroughs in RL and learning from demonstrations literature, and this paper aims at making the most of these promising findings in order to solve the visual pouring task.

III. SELF-SUPERVISED APPROACHING LEARNING METHODOLOGY

The goal of the proposed method consists in teaching a robots for a complex activity, particularly visual pouring by specifying only the goal's circumstances. Based on the RGB image of the state, the robot evaluates each state as a goal or not-goal and assigns a score to it. A self-supervised data collection approach is used to train this classification network which serves as the reward function. A soft actor-critic is used to learn the policy and is employed for action selection after the reward function has been trained. For the training stage of this RL algorithm, the trained classifier is used as the reward function. The content of this section falls into two parts:

- The logic of the self-supervised reward function training, goal conditions definition, data collecting, network architecture and the employed deep metric learning method;
- The reasoning applied for the soft actor-critic method, action-state coding, efficient exploration, network design, and training process.

Figure 3 depicts the key difference between strategies that are used in this paper, IRL and VICE method for demonstrations providing. VICE requires only the final goal state as observation, whereas IRL requires the entire trajectory as observation and can only recreate similar actions in the testing phase. For VICE, the time of convergence may be slower

than IRL, but the effort of data gathering and reward function design is reduced significantly. Finally, the suggested approach aims at decreasing the data collection and training convergence time, as well as human effort and supervision by getting the target conditions and reproducing states which lead to the goal easily.

A. Reward Function Approximator

A vital stage in this research is to create a reliable and accurate reward network for the purposes of this study, i.e., the pouring task accomplished by a NAO H25 robot. It is explained how to train a reward network using pseudo-code 1. For data collecting, two sets are initialized D^+ and D^- and then, for each episode robot's hand and target bowl are initialized in a random position and the robot moves its hand to an arbitrary point close to the bowl. At this moment the camera image is saved into a buffer and the robot twists its arm if more than a cut-off count of spheres lied into the target container the buffered image is appended to D^+ , and if not the is append to D^- . After ending all episodes the neural network is trained using D^+ and D^- labeled images.

Algorithm 1: Data collecting and train reward network

```

1 Start simulation and data gathering module
2 initialize dataset  $\mathbb{D} : \{D^-, D^+\}$ 
3 for  $i$  in  $\{1, \dots, N\}$  do
4   Move the glass to an arbitrary point around the
     bowl;
5   Save image  $\mathbb{I}$  in buffer ;
6   Twist robot hand to spill content of glass to bowl;
7   if Content lied in bowl then
8     Append image  $\mathbb{I}$  to  $D^+$  ;
9   else
10    Append image  $\mathbb{I}$  to  $D^-$  ;
11  end
12 end
13 Train Deep-RBF classifier using labeled images;
```

Goal condition defining is the first step for training reward network. In this study, conditions are defined based on the

success or failure of the next action. For training a robot which must perform the pouring task, a reward function is needed to determine the probability of success in each state based on situations that leads to easily pouring by a single simple act like wrist twisting so the reward approximator must be able to predict the next state is goal or not. For another instance, if the mission is to pick up a glass, a state is marked as a goal if the robot is able to lift the glass by raising its hand. These conditions can be verified by position data in simulation or computer vision techniques in the real world readily.

For collecting positive and negative samples, the robot hand which carries the glass is initialized in a random position, and in each episode, the robot is allowed to move its hand to an arbitrary 3D point in an area around the target bowl which is likely to be good for pouring, this step is called random guided exploration.

Now, acquired data can be used to train a CNN network. Networks with Softmax and cross-entropy loss give high scores to inputs that are in regions with little or no training data [20]. As a result, a deep-RBF network, shown in Figure 4, is chosen in the reward function design to avoid network sensitivity to unobserved input and RL algorithm deception. This architecture consists of a single class (positive class) and rejection option and the hyper-parameter λ stands for the distance threshold and can be modified in training or test phases. The loss function which is used in training consists of two terms which are designed for positive and reject classes:

$$J_{ML} = \sum_{i=1}^N \left(yd(x_i) + (1-y)\max(0, \lambda - d(x_i)) \right)$$

In the above equation, $d(x_i)$ is the distance of sample x_i for positive class and λ represents the distance threshold and y is 1 for positive samples and 0 for negative ones.

B. Soft Actor-Critic

Soft Q-Network, and Policy Network are the two neural networks which make up the soft actor-critic algorithm. Each of these networks uses a CNN to analyze the state image, resulting in a fully connected layer which combines action and state data. The state image is the input to the policy and the state and action are the input to the Q-network in order to estimate their Q-value. Algorithm 2 contains the pseudo-code for the RL algorithm. In this algorithm, k_{random} and k_{guided} are respectively the number of steps which the robot tries random actions and random guided action which probably are closer to the desire action. Moreover, k_{eval} represents the number of evaluation episodes which are performed after each M training episode and π_{θ} is the learned policy and $\mathcal{R}$ is replay buffer which is used for network training by batch random sampling. Before launching the main RL training phase and receiving positive reward, the robot performs some random exploration and then some guided exploration, in which the robot hand is pushed into a new location which is suitable for pouring. This act may aid to ameliorate the speed of convergence.

Algorithm 2: Train Soft Actor-Critic (SAC)

```

1 Initialize  $M, k_{\text{random}}, k_{\text{guided}}, k_{\text{eval}}$ 
2 Initialize policy  $\pi$ , critic  $\mathcal{Q}$ 
3 Initialize replay buffer  $\mathcal{R}$ 
4 Choose random positions for  $k_{\text{random}}$  steps
5 Do guided exploration for  $k_{\text{guided}}$  steps
6 for each iteration do
7   for each environment step do
8      $a_t \sim \pi_{\theta}(a_t|s_t)$ 
9      $s_{t+1} \sim p(s_{t+1}|s_t, a_{t+1})$ 
10     $\mathcal{R} \leftarrow \mathcal{R} \cup (s_t, a_t, r(s_t, a_t), s_{t+1})$ 
11  end
12  for each gradient step do
13    Sample from  $\mathcal{R}$ 
14    Update  $\pi$  and  $\mathcal{Q}$  according to [21]
15  end
16  if  $s \bmod(M) == 0$  then
17    for  $k_{\text{eval}}$  steps do
18       $a_t \sim \pi_{\theta}(a_t|s_t)$ 
19      Do action  $a_t$ 
20    end
21    log average collected reward
22 end
```

For defining action-space, some ideas were tested which can be addressed the experiments section, and finally, the position-based action-space is chosen. In each step, the stochastic policy generates three Gaussian distributions that represent the position in the 3D coordinates system. This simplified action helps the RL algorithm to converge faster. In training phase, agent generates samples from these distributions as action but in evaluation or testing phases the mean value of distributions are chose.

IV. RESULTS OF TRAINING AND EVALUATION IN THE SIMULATION ENVIRONMENT

This study uses CoppeliaSim as a simulation environment for both training and evaluation phases. To do so, a CoppeliaSim environment is prepared as the set up to test the proposed approach, which allowing a NAO H25 robot to interact with items on a table. In the NAO H25 robot's hand is a glass containing some little spheres, and on the table is an empty bowl. The bowl, as well as the robot hand, are randomly initialized around a certain location in each episode.

The robot performed some random guided exploration to train the reward network, twisting its hand in a random 3D position near the bowl at each step. The glass holds 15 spheres, and if more than 10 spheres are found in the container during a trial, the frame before the robot arm twists is saved as a positive sample, and vice versa. This method yielded 3700 images, including 1700 negative samples. As depicted in Figure 4, the network was trained for 100 epochs with $\lambda = 450$ and obtained 98 percent accuracy on this training set and

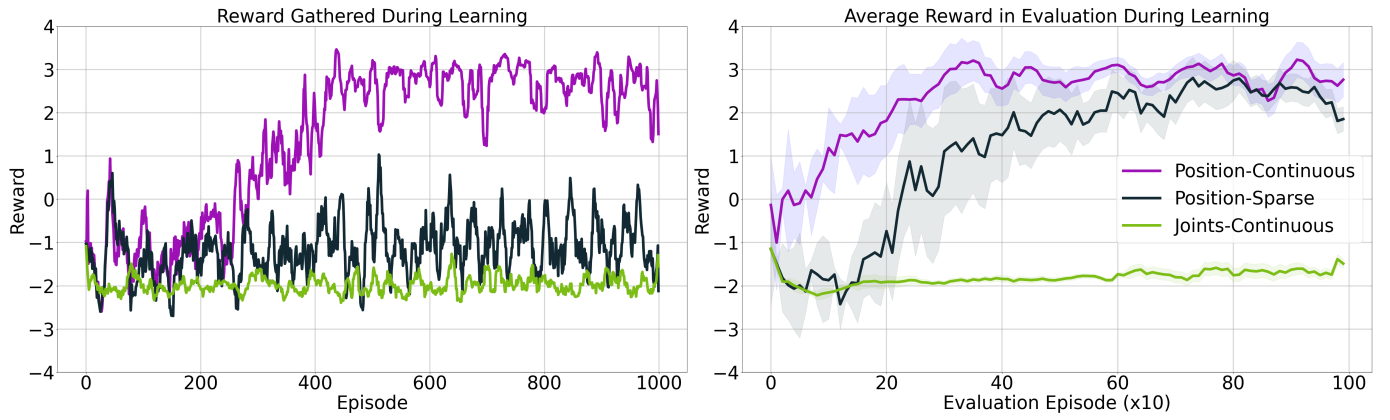


Fig. 5. In 1k episodes, the gathered reward of the pouring task is presented. For action coding, two forms of action, *Position* and *Joints*, are employed, as well as two types of reward, *Continuous* and *Sparse*. In 1k episodes, the best performance was for *Position-Continuous*, and *Joints-Continuous* did not converge.

93 percent success rate in evaluation phase after training the policy. On a test set including 1500 pictures with 500 negative samples the accuracy of best goal classifier reach to 83 and the difference between the success rate and the goal classification comes from the power of stochastic policy and Deep RL approaches which learn the most confident parameters to generate Gaussian distribution in each step which individual CNN is not able to perform correctly this task compared to the Deep RL method. The results of trained network including Deep-RBF output layer with and without augmentation and also CNN with cross-entropy loss and softmax output layer are shown in Table I. Random crop, resize, color manipulation, and jittering are used as augmentation. For training and testing the reward network, the input image has a 160x160 dimension and the embedding 1D vector that is fed to the linear layer with 800 neurons has 9248 elements.

The trained reward network is used as the main reward source for training the RL policy in two ways, namely, continuous and sparse. In the continuous approach, the network's output is scaled between -3.5 and 3.5 and directly fed to the RL algorithm, while in the sparse approach, if the score was more than 1, the reward was 1 and if it was less than 1, the reward was 0. The threshold 1 is gained practically by evaluating the reward network and is shown that samples with a reward of more than 1 have more than a 90 percent chance to complete the task successfully. Unlike the reward network, the input of policy and critic networks has a 64x64 dimension and the 1D embedding vector of them has 800 elements.

Three alternative attempts are applied for action-space. To begin, the rotation of robot's right hand joints are used as action-space, which are scaled between 0 and 1. This is a general methodology which is usually utilized in robotics RL challenges, but in the case of this paper where 1k episodes should be studied, this method fails to converge and will not result in good rewards. As a remedy, one can resort to position based action-space in the provided environment, and in each step, the robot selects a point in 3D space and moves its hand to that point. This technique, which is combined with

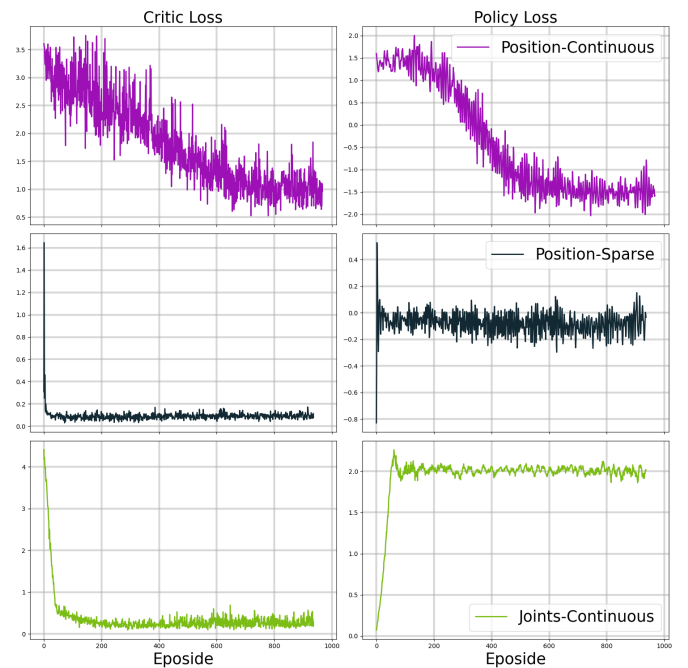


Fig. 6. Loss of critic (left) and policy (right) networks.

continuous reward, received the best reward throughout the evaluation, which was performed every 10 steps. Another coding technique used in this study was differential 3D movement, in which the robot moves its hand depending on a 3-elements vector provided by the policy. However, the results of this method did not make sense, and the policy produced the same output for different states.

The collected reward in learning and evaluation phases for joint-based action-space and position-based action-space combining continuous and sparse rewards are illustrated in Figure 5. After each 10 episodes in learning process, evaluation is ran 5 times, and the reward values, as well as the standard deviation which is shown in this figure, are derived from these runs. Also, critic and policy losses during training

TABLE I
ACCURACY OF DIFFERENT TRAINED REWARD NETWORKS.

Output layer	Augmentation	Accuracy
Softmax	✗	73.3
Softmax	✓	81.2
Deep-RBF	✗	72.8
Deep-RBF	✓	83.1

TABLE II
SUCCESS RATE OF TESTED SCENARIOS AFTER 1K EPISODES.

Method	Success Rate	Is Convergence
Position-Continuous	93.7	✓
Position-Sparse	78.2	✓
Joints-Continuous	0	✗

are presented in Figure 6 which for position-continuous both critic and policy losses decreased gradually over time but for position-sparse and joints-continuous the critic loss decreased suddenly and actor loss increased in the initial steps and both continued around a fixed value. This situation comes from the reward sparsity nature in position-continuous and receiving a low reward in joints-continuous approach. Success rate of trained policies with different action-space coding in a diffrenet evaluation environment is listed in Table II which validate trend of losses.

V. CONCLUSION AND REMARKS

In this work, the visual pouring task is split into several steps and the approaching step is addressed using Deep RL. The proposed self-supervised data gathering method reduced the need for expert supervision and positive samples collected only by defining the goal semantics conditions. While most studies use 3D information i.e., RGB-D images, markers, or robot joints positions, raw RGB image was the only data source is used in this study. Another contribution of this study was discussing Deep RL training efficiency which is compared with other methods based on the speed of convergence and number of needed episodes. Intuitive action-space defining helped to reduce the number of needed episodes from 100k-1M to 1k by simplifying the action-space coding. Utilizing a Deep-RBF output layer is another novelty that does not get any attention from the RL community but these types of architectures can lead to a more robust learning process and prevent policy to fool the reward network and finally, the success rate of 93.7 is achieved by the proposed method in this paper after 1k training episodes. This study proposed a pipeline to teach the approaching phase to a NAO H25 robot with minimal human effort and in further works, other phases of the pouring task can be addressed by similar reasoning. Moreover, adding human hints and commands in a human-robot interaction setup can be done by combining soft and hard information fusion techniques with the proposed approach in further studies.

REFERENCES

- [1] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1334–1373, 2016.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al., "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] C. Do and W. Burgard, "Accurate pouring with an autonomous robot using an rgb-d camera," in *International Conference on Intelligent Autonomous Systems*, pp. 210–221, Springer, 2018.
- [4] C. Finn, T. Yu, T. Zhang, P. Abbeel, and S. Levine, "One-shot visual imitation learning via meta-learning," in *Conference on Robot Learning*, pp. 357–368, PMLR, 2017.
- [5] H.-Y. Tung, A. W. Harley, L.-K. Huang, and K. Fragkiadaki, "Reward learning from narrated demonstrations," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7004–7013, 2018.
- [6] A. Singh, L. Yang, K. Hartikainen, C. Finn, and S. Levine, "End-to-end robotic reinforcement learning without reward engineering," *arXiv preprint arXiv:1904.07854*, 2019.
- [7] D. S. Leiden, *Cognitive reasoning for compliant robot manipulation*. Springer, 2019.
- [8] P. Gajewski, P. Ferreira, G. Bartels, C. Wang, F. Guerin, B. Indurkha, M. Beetz, and B. Śnieżyński, "Adapting everyday manipulation skills to varied scenarios," in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 1345–1351, IEEE, 2019.
- [9] S. Ha, P. Xu, Z. Tan, S. Levine, and J. Tan, "Learning to walk in the real world with minimal human effort," *arXiv preprint arXiv:2002.08550*, 2020.
- [10] H.-Y. Li, T. Nuradha, S. A. Xavier, and U.-X. Tan, "Towards a compliant and accurate cooperative micromanipulator using variable admittance control," in *2018 3rd International Conference on Advanced Robotics and Mechatronics (ICARM)*, pp. 230–235, IEEE, 2018.
- [11] H. Hosseini, M. T. Masouleh, and A. Kalhor, "Improving the successful robotic grasp detection using convolutional neural networks," in *2020 6th Iranian Conference on Signal Processing and Intelligent Systems (ICSPIS)*, pp. 1–6, IEEE, 2020.
- [12] A. Chan, E. A. Croft, and J. J. Little, "Modeling nonconvex workspace constraints from diverse demonstration sets for constrained manipulator visual servoing," in *2013 IEEE International Conference on Robotics and Automation*, pp. 3062–3068, IEEE, 2013.
- [13] B. V. Adorno, *Two-arm manipulation: From manipulators to enhanced human-robot collaboration*. PhD thesis, Université Montpellier II-Sciences et Techniques du Languedoc, 2011.
- [14] Z. Zhu and H. Hu, "Robot learning from demonstration in robotic assembly: A survey," *Robotics*, vol. 7, no. 2, p. 17, 2018.
- [15] P.-J. Hwang, C.-C. Hsu, and W.-Y. Wang, "Development of a mimic robot: Learning from human demonstration to manipulate a coffee maker as an example," in *2019 IEEE 23rd International Symposium on Consumer Technologies (ISCT)*, pp. 124–127, IEEE, 2019.
- [16] J. Fu, A. Singh, D. Ghosh, L. Yang, and S. Levine, "Variational inverse control with events: A general framework for data-driven reward definition," *arXiv preprint arXiv:1805.11686*, 2018.
- [17] A. S. Chen, H. Nam, S. Nair, and C. Finn, "Batch exploration with examples for scalable robotic reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4401–4408, 2021.
- [18] M. Chi, Y. Yao, Y. Liu, Y. Teng, and M. Zhong, "Learning motion primitives from demonstration," *Advances in Mechanical Engineering*, vol. 9, no. 12, p. 1687814017737260, 2017.
- [19] C. Dong, M. Takizawa, S. Kudoh, and T. Suehiro, "Precision pouring into unknown containers by service robots," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5875–5882, IEEE, 2019.
- [20] P. H. Zadeh, R. Hosseini, and S. Sra, "Deep-rbf networks revisited: Robust classification with rejection," *arXiv preprint arXiv:1812.03190*, 2018.
- [21] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*, pp. 1861–1870, PMLR, 2018.